

FASE 0 · O AGENTE EM CÓDIGO PURO

Aula 0.5

As três ferramentas reais do domínio

CRM, agenda e tickets sobre SQLite. E a descoberta que muda a qualidade de um agente: descrição de ferramenta é prompt.

Série: Engenharia de IA Aplicada · Agentes e RAG em Produção

Projeto: Núcleo de Agentes Operacionais

Duração de estudo: cerca de 1h30 · 25 min de teoria e 55 de código

Pré-requisito: Aulas 0.1 a 0.4 (tag **aula-0.4**)

1. Objetivo de aprendizagem

Ao final desta aula você será capaz de projetar ferramentas que um modelo escolhe corretamente, aplicando quatro regras de design de schema, e terá o Núcleo operando sobre dados reais persistidos em banco.

2. Por que esta aula importa

Até aqui as ferramentas foram mocks de duas linhas, e o agente escolheu certo porque só havia duas opções óbvias. Com três ferramentas que se parecem, e depois com as dez ou quinze de um sistema real, a escolha deixa de ser óbvia.

E aqui está o ponto que separa um agente que funciona de um que erra o tempo todo, e ele não é sobre código: **o nome, a descrição e os parâmetros de uma ferramenta mudam drasticamente a acurácia de seleção**. O modelo escolhe lendo esse texto. Se o texto for ambíguo, a escolha será ambígua, e nenhuma engenharia de laço conserta isso.

A FRASE DA AULA

Descrição de ferramenta é prompt. Não é documentação, não é comentário, não é detalhe. É o texto que o modelo lê no momento de decidir, e ele compete com os outros textos que você mandou. Escrever schema é engenharia de prompt com outro nome.

3. Teoria

3.1 A regra de arquitetura: negócio no prompt, capacidade na ferramenta

Antes das quatro regras de design, uma decisão estrutural que vale o curso inteiro.

`abrir_ticket` não decide **quando** um ticket deve ser aberto. Ela sabe abrir. Quem decide é o agente, orientado pela instrução de sistema. Se você colocar dentro da ferramenta um `if` que só abre ticket para cliente enterprise, acabou de criar uma ferramenta que serve para um cliente só.

O Núcleo precisa ser replicável: o mesmo código, outra base, outras regras. Por isso a política vive no prompt, que troca por configuração, e a capacidade vive na ferramenta, que é código.

A fronteira nem sempre é óbvia. Recusar agendar em horário já ocupado é **capacidade**, porque a agenda fisicamente não comporta dois compromissos no mesmo slot. Recusar agendar com lead não qualificado é **negócio**, e muda de cliente para cliente. A pergunta que resolve: isso seria verdade em qualquer empresa que usasse esta ferramenta?

3.2 As quatro regras de design de ferramenta

Regra 1. Nome no formato verbo mais objeto. `agendar_reuniao`, não `reuniao` nem `processar`. O nome já deve dizer a ação, porque é a primeira coisa que o modelo lê.

Regra 2. A descrição diz quando usar, não só o que faz. Compare:

Descrição	
Ruim	"Abre um ticket de suporte."

Boa	"Abre um chamado de suporte técnico. Use quando o lead relatar problema, erro ou indisponibilidade no produto. Não use para dúvidas comerciais ou pedidos de proposta."
------------	---

A segunda orienta a decisão e, principalmente, **delimita**: dizer quando não usar é tão útil quanto dizer quando usar, porque é assim que o modelo distingue ferramentas parecidas.

Regra 3. Enum em vez de texto livre. Se a prioridade só pode ser baixa, média ou alta, diga isso no schema. Texto livre convida o modelo a escrever "urgentíssima", e aí você tem um erro de validação que poderia simplesmente não existir. Enum fecha o vocabulário na origem.

Regra 4. Retorne texto rico, não código de status. O resultado da ferramenta é lido pelo modelo, e vira a observação da próxima volta do laço. `{"ok": true}` não ajuda ninguém. Compare:

	Retorno	O que o agente consegue fazer
Ruim	<code>{"erro": "conflito"}</code>	Repetir a mesma chamada e entrar em ciclo.
Bom	"Horário indisponível: já existe reunião em 14/08 às 10:30. Sugira outro horário ao lead."	Propor outro horário e seguir a tarefa.

Repare que o retorno bom **diz o próximo passo**. É assim que um agente se recupera sozinho, e é a semente da resiliência da aula 0.6.

3.3 Uma consequência: o registro precisa crescer

Na aula 0.4, o `MAPA_TIPOS` cobria quatro tipos simples, e eu disse que tipos novos entrariam quando o projeto precisasse. As regras 2 e 3 são exatamente essa necessidade: precisamos de **enum** e de **descrição por parâmetro**, e o registro ainda não expressa nenhum dos dois.

Isso é design incremental funcionando, não um remendo. A extensão é pequena e a fonte da verdade continua sendo a função.

3.4 Por que SQLite

Zero dependência, roda em qualquer máquina, e é substituível. Na Fase 4, trocar por Postgres é reescrever o módulo de banco, não as ferramentas. A fronteira entre dados e capacidade fica no `banco.py`.

4. Mão na massa

Esta aula cria `nucleo/banco.py` e `nucleo/ferramentas.py`, estende `nucleo/registro.py` e acrescenta os testes. O `agente.py` continua intocado desde a aula 0.3.

Passo 1 — Estender o registro: Literal vira enum

O que e por quê: a regra 3 exige enum. Em Python, o jeito idiomático de dizer "só estes valores" é `typing.Literal`, e ele é legível por introspecção.

```
def _tipo_json(anotacao, nome_ferramenta, nome_param):
    """Traduz o type hint para JSON Schema.

    Aula 0.5: além dos quatro tipos simples, aceitamos Literal, que
    vira 'enum'. Enum fecha o vocabulário: em vez de o modelo inventar
```

```

"urgentissima", ele so pode escolher entre os valores que existem.
"""

if typing.get_origin(annotacao) is typing.Literal:
    valores = list(typing.get_args(annotacao))
    # 0 tipo do enum e o tipo do primeiro valor (na pratica, string).
    base = MAPA_TIPOS.get(type(valores[0]), "string")
    return {"type": base, "enum": valores}

tipo = MAPA_TIPOS.get(annotacao)
if tipo is None:
    raise TypeError(
        f"Tipo nao suportado em '{nome_ferramenta}.{nome_param}': "
        f"{annotacao}. Use str, int, float, bool ou Literal."
    )
return {"type": tipo}

```

O que faz: se a anotação for um `Literal`, extraímos os valores e geramos `enum` no schema. Qualquer outro tipo segue pelo `MAPA_TIPOS` de antes.

Passo 2 — Estender o registro: descrição por parâmetro

O que e por quê: a regra 2 vale também para cada argumento. Dizer que a data usa o formato AAAA-MM-DD é a diferença entre receber `2026-08-14` e receber "14 de agosto". A fonte continua sendo a docstring, agora com um bloco `Args:` no estilo Google.

```

def _partir_docstring(doc):
    """Separa a docstring em (descricao da ferramenta, descricao por parametro).

    Formato esperado, estilo Google:

        Resumo do que a ferramenta faz e quando usar.

        Args:
            email: Email do lead, ex: nome@empresa.com
            plano: Plano atual do lead

    A parte antes de 'Args:' e o que o modelo le para DECIDIR usar a
    ferramenta. As linhas de Args descrevem cada argumento, e entram no
    JSON Schema. Os dois textos sao prompt: e deles que sai a acuracia
    de selecao.
    """
    if "Args:" not in doc:
        return doc.strip(), {}

    cabeca, _, cauda = doc.partition("Args:")
    descricoes = {}
    for linha in cauda.strip().splitlines():
        linha = linha.strip()
        if not linha or ":" not in linha:
            continue
        nome, _, texto = linha.partition(":")
        descricoes[nome.strip()] = texto.strip()
    return cabeca.strip(), descricoes

```

O que faz: separa o resumo (que o modelo lê para decidir usar a ferramenta) das descrições de cada argumento (que entram nas `properties` do schema). No decorator, cada propriedade ganha sua `description` quando existir.

Passo 3 — O banco

O que e por quê: três tabelas, dados de exemplo, e uma decisão que parece detalhe e não é.

```
CAMINHO_BANCO = Path("flowdesk.db")

def conectar():
    """Abre conexao com o banco, devolvendo linhas acessiveis por nome."""
    con = sqlite3.connect(CAMINHO_BANCO)
    # row_factory: cursor devolve objetos indexaveis por nome de coluna
    # (linha["email"]) em vez de tuplas posicionais.Codigo mais legivel.
    con.row_factory = sqlite3.Row
    return con
```

O que faz: o caminho do banco é **variável de módulo**, não constante enterrada nas funções. É isso que permite os testes apontarem para um banco temporário sem tocar no seu arquivo de trabalho. Decisão de testabilidade tomada na hora de escrever, não depois.

O `criar_schema()` usa `CREATE TABLE IF NOT EXISTS` e `INSERT OR IGNORE`: é idempotente, roda quantas vezes quiser sem duplicar nem quebrar.

Passo 4 — A primeira ferramenta: CRM

O que e por quê: consulta e atualização na mesma ferramenta, porque para o modelo são a mesma intenção: mexer no cadastro do lead. Veja as quatro regras aplicadas.

```
@ferramenta
def buscar_atualizar_crm(email: str, status: str = "",
                        notas: str = "") -> str:
    """Consulta um lead no CRM pelo email e, opcionalmente, atualiza.

    Use SEMPRE antes de qualquer outra acao sobre o lead, para
    confirmar que ele existe e conhecer plano, MRR e status. Se status
    ou notas forem informados, o cadastro e atualizado; se nao, a
    ferramenta apenas consulta.

    Args:
        email: Email do lead, ex: nome@empresa.com
        status: Novo status do lead. Deixe vazio para apenas consultar.
        notas: Observacao a acrescentar ao cadastro. Vazio nao altera.
    """
    con = conectar()
    cur = con.cursor()

    linha = cur.execute(
        "SELECT * FROM leads WHERE email = ?", (email,)).fetchone()

    if linha is None:
        con.close()
        # Texto, nao codigo de erro: quem le isto e o modelo.
        return f"Lead nao encontrado no CRM: {email}"

    alteracoes = []
    if status:
        cur.execute("UPDATE leads SET status = ? WHERE email = ?",
                    (status, email))
        alteracoes.append(f"status agora e '{status}'")
    if notas:
        cur.execute("UPDATE leads SET notas = ? WHERE email = ?",
                    (notas, email))
        alteracoes.append("notas atualizadas")
```

```

if alteracoes:
    con.commit()
    linha = cur.execute(
        "SELECT * FROM leads WHERE email = ?", (email,)).fetchone()
    con.close()

# Resultado RICO: o modelo compoe a resposta a partir deste texto.
# Um {"ok": true} nao ajudaria ninguem.
texto = (f"{linha['nome']} ({linha['empresa']}), plano "
        f"{linha['plano']}, MRR {linha['mrr']}, status "
        f"{linha['status']}".)
if linha["notas"]:
    texto += f" Notas: {linha['notas']}"
if alteracoes:
    texto += " Atualizado: " + ", ".join(alteracoes) + "."
return texto

```

O que faz: o resumo diz quando usar ("SEMPRE antes de qualquer outra ação"), o bloco `Args` descreve cada argumento, e o retorno é uma frase que o modelo consegue usar para compor a resposta. Lead inexistente devolve texto, não exceção: é observação, e o agente contorna.

Passo 5 — A ferramenta com enum

O que e por quê: `abrir_ticket` é onde a regra 3 aparece, e onde a delimitação da regra 2 evita que ela seja confundida com `agendar_reuniao`.

```

@ferramenta
def abrir_ticket(email: str, assunto: str, descricao: str,
                prioridade: Literal["baixa", "media", "alta"]) -> str:
    """Abre um chamado de suporte tecnico para um lead ou cliente.

    Use quando o lead relatar problema, erro ou indisponibilidade no
    produto. Nao use para duvidas comerciais ou pedidos de proposta.

    Args:
        email: Email de quem relatou o problema
        assunto: Titulo curto do chamado
        descricao: O problema relatado, com os detalhes disponiveis
        prioridade: Urgencia do chamado
    """
    con = conectar()
    cur = con.cursor()

    if cur.execute("SELECT 1 FROM leads WHERE email = ?",
                  (email,)).fetchone() is None:
        con.close()
        return f"Nao foi possivel abrir o ticket: {email} nao esta no CRM."

    cur.execute(
        "INSERT INTO tickets (email_lead, assunto, descricao, prioridade) "
        "VALUES (?, ?, ?, ?)", (email, assunto, descricao, prioridade))
    con.commit()
    numero = cur.lastrowid
    con.close()

    return (f"Ticket #{numero} aberto para {email}, prioridade "
            f"{prioridade}. Assunto: {assunto}.")

```

E este é o schema que o registro gera sozinho, sem uma linha escrita à mão:

```

"name": "abrir_ticket",
"description": "Abre um chamado de suporte tecnico para um lead ou cliente.\n\nUse quando o lead relatar problema, erro ou indisponibilidade no produto. Nao use para duvidas comerciais ou pedidos de proposta.",
"input_schema": {
  "type": "object",
  "properties": {
    "email": {"type": "string", "description": "Email de quem relatou o problema"},
    "assunto": {"type": "string", "description": "Titulo curto do chamado"},
    "descricao": {"type": "string", "description": "O problema relatado, com os detalhes disponiveis"},
    "prioridade": {"type": "string", "enum": ["baixa", "media", "alta"], "description": "Urgencia do chamado"}
  },
  "required": ["email", "assunto", "descricao", "prioridade"]
}

```

O que faz: o `enum` fecha o vocabulário da prioridade e cada argumento carrega sua descrição. Tudo derivado da função. A ferramenta `agendar_reuniao`, que fica entre as duas no arquivo, segue exatamente o mesmo padrão.

Passo 6 — Ver funcionando, com regra de negócio no prompt

O que e por quê: agora o rascunho ganha uma instrução de sistema, e é ali que mora a política comercial. Compare com as ferramentas: nenhuma delas sabe o que é um lead qualificado.

```

# scratch/rascunho_0_5.py (descartavel: o .gitignore ignora scratch/)
from nucleo import banco, ferramentas # importar REGISTRA as ferramentas
from nucleo.registro import schemas, executores
from nucleo.agente import rodar_agente

banco.criar_schema() # cria o flowdesk.db com os 3 leads

SISTEMA = """Voce e o assistente comercial do FlowDesk, um SaaS B2B.

Antes de qualquer acao sobre um lead, consulte o CRM para confirmar que
ele existe. Considere um lead qualificado quando o plano for pro ou
enterprise, ou o MRR passar de 300; nesse caso, atualize o status e
proponha uma reuniao. Problemas tecnicos viram ticket, nunca reuniao."""

pergunta = ("A Ana do TechFlow escreveu: o login pelo SSO esta dando erro 500 "
            "desde ontem, e ela quer conversar sobre migrar de plano.")

exe = rodar_agente(pergunta, schemas(), executores(), system=SISTEMA)
print(exe.parou_por, exe.iteracoes)
print(exe.resposta)

```

Resultado esperado: o agente consulta o CRM, abre um ticket de prioridade alta pelo erro de login, e agenda a conversa sobre o plano. Três ferramentas, uma pergunta. Agora mude uma frase do `SISTEMA`, por exemplo exigindo MRR acima de 1000 para qualificar, e rode de novo: o comportamento muda sem você tocar em uma linha de Python. Isso é a regra de arquitetura funcionando.

Passo 7 — Os testes

O que e por quê: ferramenta é Python comum, porque o decorator não a modifica. Chamamos direto e conferimos o efeito no banco. A fixture garante que cada teste rode contra um banco novo.

```
@pytest.fixture(autouse=True)
def banco_temporario(tmp_path, monkeypatch):
    """Aponta CAMINHO_BANCO para um arquivo novo a cada teste.

    tmp_path é uma pasta temporária por teste, do próprio pytest.
    monkeypatch desfaz a troca no fim. E por isso que o caminho do
    banco é variável de módulo, e não constante dentro das funções.
    """
    monkeypatch.setattr(banco, "CAMINHO_BANCO", tmp_path / "teste.db")
    banco.criar_schema(semear=True)
    yield
```

E o teste que cobra a regra 4, que é a mais fácil de esquecer:

```
# ----- 5. conflito de horario orienta o proximo passo -----

def test_horario_ocupado_sugere_alternativa():
    ferramentas.agendar_reuniao(
        "carlos@datalog.io", "2026-08-14", "10:30", "Demo")
    saida = ferramentas.agendar_reuniao(
        "bia@scalelab.com", "2026-08-14", "10:30", "Renovacao")

    assert "indisponivel" in saida.lower()
    # O texto precisa DIZER o que fazer: e assim que o agente se
    # recupera sozinho, em vez de repetir a mesma chamada.
    assert "outro horario" in saida.lower()
```

Resultado esperado: `pytest` mostra `32 passed, 2 deselected`, somando as aulas anteriores.

Passo 8 — Commit e tag

```
pytest -v          # esperado: 32 passed, 2 deselected
git status         # .env, scratch/ e *.db NÃO podem aparecer
git add .
git commit -m "Aula 0.5: as tres ferramentas reais do dominio"
git tag aula-0.5
git push && git push --tags
```

5. Portabilidade

As quatro regras não são sobre Python nem sobre provider: valem para qualquer ferramenta que um LLM vá escolher, em qualquer framework. Quando você expuser essas mesmas ferramentas por um MCP server na Fase 2, o schema será o mesmo e as regras continuarão valendo, porque quem lê continua sendo um modelo.

A separação negócio no prompt, capacidade na ferramenta é o que vai permitir, na Fase 2, que o supervisor e os especialistas compartilhem as mesmas ferramentas com políticas diferentes.

6. Terminei quando

O critério da aula: **o agente qualifica um lead fictício usando as três ferramentas.**

- O rascunho resolve o caso da Ana abrindo ticket e agendando reunião, sem você indicar a ordem.
- Você muda o critério de qualificação no `SISTEMA` e o comportamento muda, sem editar Python.

- Tente agendar duas reuniões no mesmo horário: o agente recebe a orientação e propõe outro, em vez de repetir a chamada.
- `pytest` mostra 32 passed, 2 deselected.

7. Erros comuns e troubleshooting

- **As ferramentas não aparecem nos schemas.** Ninguém importou `nucleo.ferramentas`, então o decorator não rodou. Registro por decorator depende de importação.
- **no such table: leads.** Faltou `banco.criar_schema()` antes de usar as ferramentas.
- **O agente confunde ticket com reunião.** As descrições não estão delimitando. Acrescente o "não use para" na que estiver sendo escolhida por engano.
- **O modelo manda a data em formato livre.** A descrição do parâmetro não diz o formato. É a regra 2 aplicada ao argumento, e sem ela a ferramenta grava lixo.
- **O agente repete a mesma chamada e estoura o limite.** O retorno da ferramenta não diz o próximo passo. Regra 4.
- **O banco foi versionado.** Acrescente `*.db` ao `.gitignore`.

8. Ponte para a próxima aula

O Núcleo agora age sobre dados reais, e isso cria um problema novo: o que acontece quando o banco cai no meio de um atendimento, ou quando o modelo inventa um argumento que não existe? Na **Aula 0.6, Saída estruturada e resiliência a erro**, erro de ferramenta vira observação que o agente contorna, e a resposta final ganha um contrato validado, pronto para outro sistema consumir.